# Viva Questions & Answers

Intelligent Diabetes Risk Predictor — CS619 Final Project

Prepared: June 24, 2026

---

## Project Overview

### Q1. What is your project about?

A: It is a web-based machine learning decision-support system that predicts diabetes risk from clinical and lifestyle data. Patients enter health metrics and get a risk score with explanations and recommendations. Doctors review assigned patients, and admins manage users, datasets, model training, and the system.

### Q2. Why did you choose this topic?

A: Diabetes is a growing global health problem, and early risk detection helps prevention. Machine learning can analyze multiple health factors together. A web app makes this accessible to patients and useful for doctors as a decision-support tool.

### Q3. What problem does your system solve?

A: It helps users understand their diabetes risk early, tracks health over time, explains which factors contribute most, and gives personalized recommendations. It also supports clinical workflows through doctor notes and admin oversight.

### Q4. Is this a diagnostic system?

A: No. It is a decision-support system. It estimates risk based on patterns in historical data. Final diagnosis must be done by a qualified doctor using proper clinical tests.

## Technology & Architecture

### Q5. Which technologies did you use and why?

A: Python and Flask for the backend because they are lightweight and good for ML integration. SQLAlchemy handles the database. scikit-learn is used for ML. Bootstrap and Chart.js provide the UI. joblib saves trained models. Gunicorn and Docker are used for deployment on Render.

### Q6. Explain your system architecture.

A: It is a three-tier monolithic web application: Presentation layer (HTML templates, CSS, Chart.js), Application layer (Flask routes, forms, authentication, business logic), and Data/ML layer (database, dataset files, trained models, and ML pipeline).

### Q7. What is the role of Flask blueprints in your project?

A: Blueprints organize routes into modules: auth for login/register, main for patient features, admin for administration, and provider for doctor features. This keeps the code modular.

### Q8. What is the application factory pattern?

A: In create_app(), the Flask app is created and configured in one place—database, login manager, blueprints, migrations, and seed data. This makes testing and deployment easier.

### Q9. How do you run the project locally?

A: Install requirements, run setup_data.py to download the dataset, train_initial.py to train models, then run.py to start the server at http://localhost:5000.

## Database Design

### Q10. Which database do you use?

A: SQLite for local development and PostgreSQL in production via DATABASE_URL on Render or Neon.

### Q11. Name the main database tables.

A: users, health_records, predictions, model_metrics, training_jobs, feedbacks, provider_patients, education_resources, clinical_notes, audit_logs, and report tables for admin-doctor-patient communication.

### Q12. What is the relationship between HealthRecord and Prediction?

A: Each health record stores patient vitals at a point in time. A prediction is generated from that record and stores the model output—probability, risk level, explanation, and recommendations.

### Q13. What is the ProviderPatient table used for?

A: It links doctors to their assigned patients so providers only see authorized patient data.

## User Roles & Features

### Q14. How many user roles are there?

A: Three: Patient, Provider (Doctor), and Admin.

### Q15. What can a patient do?

A: Register/login, enter health data, view predictions, see recommendations, track progress, submit feedback, export PDF/CSV reports, read education content, and send reports to admin.

### Q16. What can a doctor do?

A: Login with owner access code, view assigned patients, review predictions, add clinical notes, use clinical decision support, export patient reports, and review cases forwarded by admin.

### Q17. What can an admin do?

A: Manage users, assign doctors to patients, upload datasets, view EDA, train/retrain models, set production model, manage education content, review feedback, and view audit logs.

### Q18. What is RBAC?

A: Role-Based Access Control means each user role has specific permissions. Routes are protected using role_required so patients cannot access admin pages and doctors only see assigned patients.

## Machine Learning

### Q19. Which dataset did you use?

A: The Pima Indians Diabetes dataset. It has 8 input features and a binary outcome (diabetic or not).

### Q20. What are the 8 features?

A: Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, and Age.

### Q21. Which ML models did you implement?

A: Neural Network (MLP), SVM, Decision Tree, and Logistic Regression.

### Q22. Why use multiple models?

A: Different algorithms behave differently on the same data. Comparing them helps select the most reliable model.

### Q23. How do you preprocess the data?

A: Zero values in certain columns are treated as missing. Missing values are imputed with median. Features are scaled using StandardScaler. Data is split 70% train and 30% test.

### Q24. What is your ML pipeline?

A: Imputer, then Scaler, then Classifier—built as a scikit-learn Pipeline for consistent training and inference.

### Q25. How do you evaluate models?

A: Using Accuracy, Precision, Recall, F1-score, confusion matrix, and ROC-AUC.

### Q26. Which metric selects the best model?

A: F1-score, because it balances precision and recall on imbalanced data.

### Q27. Can admin override the best model?

A: Yes. Admin can manually set the production model from Model Settings.

### Q28. How is prediction done for a new patient?

A: Health form data is converted to the same feature format as training, passed through the saved pipeline, and predict_proba gives the diabetes probability.

### Q29. How do you define risk levels?

A: Low: below 35%. Moderate: 35% to 65%. High: above 65%.

### Q30. How does explainability work?

A: Two methods: ML feature importance from model coefficients or tree importances, and clinical threshold checks such as glucose above 140 or BMI above 30.

### Q31. How are recommendations generated?

A: Based on risk level, probability, and explanation factors—for example diet advice for high BMI.

### Q32. How do you save and load models?

A: Trained pipelines are saved using joblib in saved_models/ and loaded at prediction time with in-memory caching.

### Q33. What is model retraining?

A: When patients submit feedback with actual outcomes, admin can retrain using the original dataset plus verified feedback rows.

### Q34. Why is blood pressure converted before prediction?

A: The UI collects systolic and diastolic separately, but the dataset uses one BloodPressure feature. The system computes mean arterial pressure: (2 x systolic + diastolic) / 3.

## Security

### Q35. How are passwords stored?

A: Passwords are hashed using Werkzeug generate_password_hash, not stored in plain text.

### Q36. What is CSRF protection?

A: Flask-WTF adds CSRF tokens to forms so malicious sites cannot submit fake requests.

### Q37. What is the owner access code?

A: An extra security layer for admin and doctor login beyond username and password.

### Q38. How do you prevent brute-force login?

A: Rate limiting allows only 5 failed login attempts per IP and username within 5 minutes.

### Q39. What is an audit log?

A: A record of important actions with user, action, timestamp, and IP address for accountability.

## Testing & Deployment

### Q40. How did you test the project?

A: Using pytest unit tests in the tests/ folder and a manual test plan in docs/TEST_PLAN.md.

### Q41. How is the app deployed?

A: Using Docker and Render. render.yaml defines the web service. Gunicorn serves the Flask app.

### Q42. What happens on Render free tier?

A: The app sleeps after about 15 minutes of inactivity. First visit after sleep may take 30–60 seconds.

### Q43. Why use PostgreSQL in production?

A: SQLite data on Render can be lost on redeploy. PostgreSQL gives persistent storage.

## EDA & Reports

### Q44. What is EDA in your project?

A: Exploratory Data Analysis—summary statistics, correlation heatmap, histograms, and outcome distribution plots.

### Q45. What reports can users export?

A: Patients export PDF and CSV reports. Doctors export reports for assigned patients. Admin can export EDA as PDF.

### Q46. What is longitudinal tracking?

A: The progress page shows how a patient's risk and health metrics change over multiple health record entries.

## Common Technical Questions

### Q47. What is Flask-Login?

A: A Flask extension that manages user sessions—login, logout, and current_user across requests.

### Q48. What is WTForms used for?

A: Form creation and server-side validation before saving data or running prediction.

### Q49. What is a confusion matrix?

A: A table showing True Positives, True Negatives, False Positives, and False Negatives.

### Q50. Difference between precision and recall?

A: Precision: of predicted positives, how many were correct. Recall: of actual positives, how many the model found.

### Q51. Why use StandardScaler?

A: Features like age and glucose have different scales. Scaling helps SVM and Neural Networks perform better.

### Q52. What is overfitting?

A: When a model learns training data too closely and performs poorly on new data. Reduced using train-test split and early stopping.

### Q53. What is joblib?

A: A Python library to serialize and load scikit-learn models efficiently.

### Q54. What is Gunicorn?

A: A production WSGI server that runs the Flask app, unlike Flask's built-in development server.

## Limitations & Future Work

### Q55. What are the limitations of your project?

A: Based on a small specific dataset, only 8 features, not a medical diagnosis tool, free hosting has delays, and no mobile app or EHR integration yet.

### Q56. How can you improve the project?

A: Use larger datasets, add SHAP/LIME explainability, integrate with hospital systems, add alerts, implement MFA for staff, and use paid hosting.

### Q57. How is your project different from a simple ML script?

A: It is a complete system with user roles, database, security, explainability, recommendations, progress tracking, admin model management, feedback retraining, and deployment.

## Quick Revision

### Q58. Summarize the project in one minute.

A: Intelligent Diabetes Risk Predictor is a Flask web app with scikit-learn ML. It uses the Pima Indians dataset with 4 models (MLP, SVM, Decision Tree, Logistic Regression). Best model is selected by F1-score. Three roles: Patient, Doctor, Admin. Security includes hashing, CSRF, RBAC, rate limiting, and audit logs. Deployed on Render with Docker. It is decision support, not a diagnosis tool.

---

Supervisor: Komal Khawer — komal.khawer@vu.edu.pk